

Les listes

Exercices obligatoires

A Exercice d'entraînement au parcours itératif d'une liste chaînée

Faites le *codeRunner Liste - parcours itératifs* proposé sur moodle.

Le document *codeRunner* vous donne des conseils d'utilisation et donne le [contenu des listes testées.](#)

B Implémentation de l'interface *ListeSimple* via une liste chaînée

B1 Complétez la classe Java *ListeSimpleImpl*.

Pensez à faire des schémas !!!

Cette classe implémente l'interface *ListeSimple*. Reférez-vous à la JavaDoc de cette interface.

Testez cette classe avec la classe *TestListeSimpleImpl*.

B2 Sur moodle, répondez au questionnaire à choix multiples *ListeSimpleImpl*.

C Application Gestion des séries

On vous demande d'écrire une application qui va faciliter la tâche de la personne chargée de répartir les étudiants dans les différentes séries.

Nous n'allons pas introduire de classe *Etudiant*. Pour chaque étudiant, on ne retiendra que son nom (*String*).

C1 Vous allez compléter 3 classes.

La classe *SerieEtudiants*.

Une série d'étudiants possède un numéro et une liste d'étudiants.

Pour la liste, vous utiliserez un objet de la classe *LinkedList* de l'API Java.

La classe *ClasseEtudiants*.

La classe compte plusieurs séries d'étudiants. Celles-ci sont placées dans une table.

La classe *GestionDesSeries*.

C'est cette classe qui contient la méthode `main()`. Celle-ci-ci propose un menu.

Cette classe interagit avec l'utilisateur.

Pour éviter de devoir retaper à chaque fois beaucoup d'informations avant d'arriver à un test intéressant, on vous propose d'utiliser la classe *MonScanner* au lieu de la classe *Scanner*.

Lisez bien le pdf *monScanner* qui explique ce qu'elle fait !

Le pdf *monScanner* et le fichier texte *inputGestionDesSeries* se trouvent sur moodle dans le dossier *documents* de cette semaine.

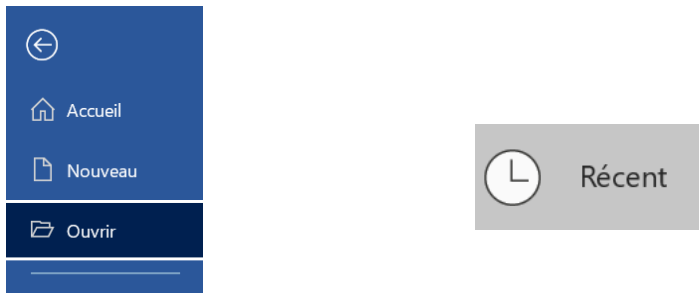
On ne vous demande pas d'écrire des classes de tests.

Mais, testez bien votre application en utilisant la classe *GestionDesSeries*.

C2 Sur moodle, répondez aux questionnaires à choix multiples *SerieEtudiants* et *ClasseEtudiants*.

D Application DocumentsLRU

Le menu *ouvrir* de Word présente les documents récemment utilisés.



Exemple :

	ExListe Documents » SD » semaine 4 - liste simple	05-02-24 16:11
	I1010 Algo - Interro 1.4.1 Documents » Algo » Interros » Interros » Interro4	05-02-24 16:10
	ExVecteurEtPileViaTable Documents » SD » semaine 1 - pile et vecteur via table	05-02-24 16:08
	ExFileViaTable Documents » SD » semaine 2 - ordre de complexité - file via tab...	05-02-24 16:07
	Janvier2024avecPC_V1 Documents » Algo » Examens » Janvier2024 » Janvier2024_V1 »...	05-02-24 16:06

L'ordre d'apparition est important. C'est le document *ExListe* qui apparaît en premier, car il est le document qui a été ouvert le plus récemment. Ensuite c'est le document *I1010 Algo – Interro 1.4.1*, l'avant dernier à avoir été ouvert et ainsi de suite.

Le fait d'ouvrir un document qui n'était pas encore présent dans la liste a comme conséquence que celui-ci apparaît maintenant en premier lieu (MRU – Most Recently Used) et que le dernier document de la liste a disparu.

Ce document est celui qui a été le moins récemment utilisé (LRU – Least Recently Used).

	ExComplexite Documents » SD » semaine 2 - ordre de complexité - file via tab...	05-02-24 16:13
	ExListe Documents » SD » semaine 4 - liste simple	05-02-24 16:11
	I1010 Algo - Interro 1.4.1 Documents » Algo » Interros » Interros » Interro4	05-02-24 16:10
	ExVecteurEtPileViaTable Documents » SD » semaine 1 - pile et vecteur via table	05-02-24 16:08
	ExFileViaTable Documents » SD » semaine 2 - ordre de complexité - file via tab...	05-02-24 16:07

Dans l'exemple, le document *ExComplexite* vient d'être ouvert. Il apparaît en premier lieu.

Le document *ExListe* apparaît maintenant en deuxième, ...

Le document *Janvier2024avecPC_V1* n'apparaît plus.

Si on ouvre un document qui se trouve déjà dans la liste, il est déplacé en premier.

Il devient le document le plus récemment utilisé.

	ExVecteurEtPileViaTable Documents » SD » semaine 1 - pile et vecteur via table	05-02-24 16:14
	ExComplexite Documents » SD » semaine 2 - ordre de complexité - file via tab...	05-02-24 16:13
	ExListe Documents » SD » semaine 4 - liste simple	05-02-24 16:11
	I1010 Algo - Interro 1.4.1 Documents » Algo » Interros » Interros » Interro4	05-02-24 16:10
	ExFileViaTable Documents » SD » semaine 2 - ordre de complexité - file via tab...	05-02-24 16:07

Dans l'exemple, le document *ExVecteurEtPileViaTable* vient d'être ouvert. Il apparaît en premier lieu.

Au cours de SD, nous allons nous intéresser à la structure de données qui permet d'optimiser le mécanisme de **LRU**.

Version semaine 4 :

La structure de données utilisée est une liste de documents (String).

Cette liste est préremplie avec autant de documents que nécessaires pour atteindre la taille que l'on a décidé préalablement et qu'on ne peut pas dépasser.

Comme, au départ, il n'y a pas de document, la liste contiendra des documents bidon aisément reconnaissables.

Lorsqu'on ouvre un document qui ne se trouve pas dans la liste, il est placé en début de liste et il faut retirer celui qui se trouve en fin de liste.

Lorsqu'on ouvre un document se trouvant dans la liste, celui-ci devient le plus récent.

Il faut donc le déplacer en tête.

La tête de la liste contient donc toujours le document le plus récemment utilisé (**M**ost **R**ecently **U**sed).

D1 Exercice préliminaire :

Après chaque ouverture d'un document, donnez le contenu de la liste.

Au départ :

doc1 doc2 doc3 doc4 doc5

ouvrir doc3

ouvrir doc4

ouvrir doc4

ouvrir doc5

ouvrir doc6

ouvrir doc3

ouvrir doc6

ouvrir doc7

Vérifiez vos réponses avec celles du document *DSol* qui se trouve sur moodle.

D2 Implémentation :

Nous vous demandons de compléter la classe *DocumentsLRU*.

Cette classe possède une liste de documents (String).

Pour la liste, vous utiliserez la classe *LinkedList* de l'API Java.

Suivez bien la *JavaDoc*.

Le jeu de tests de la classe *TestDocumentsLRU* correspond à l'exercice préliminaire.

D3 Sur moodle, répondez au questionnaire à choix multiples *DocumentsLRU*.

E Encore quelques parcours de liste

E1 Complétez la classe *ListeCaracteres*.

Toutes ces méthodes doivent avoir au maximum un ordre de complexité **O(N)**.

La classe *TestListeCaracteres*, via un menu, permet de tester ces méthodes.

Exercice supplémentaire

B3 Le bon sens nous demande de ne pas modifier une liste tout en la parcourant via un itérateur.

Par sécurité, une exception de type *ConcurrentModificationException* pourrait être levée par la méthode `next()` si la liste a été modifiée, autrement que via l'itérateur, depuis le moment où l'itérateur a été créé.

Une façon de gérer ce problème est d'introduire un numéro de version de la liste.

Chaque fois que la liste est modifiée, le numéro de version change.

Il suffit de s'assurer que ce numéro de version n'a pas changé lors de chaque appel à la méthode `next()`.

Testez cette classe avec la classe *TestListeSimpleImplSuite*.

Exercices défis

E2 Ecrivez les méthodes de la classe *ListeCaracteres* mises en défi.

B4 Acceptez les suppressions dans l'itérateur. La classe *TestIterateurRemove* permet de tester des suppressions via itérateur.